

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

Statická analýza a verifikace – projekt
CBMC – C Bounded Model Checking

3. ledna 2026

Bc. Ondřej Koumar

xkouma02@stud.fit.vutbr.cz

1 Úvod

CBMC je nástroj využívající techniky *bounded model checking* pro verifikaci programů napsaných v jazyce C. Tato technika modeluje program pomocí *bitově-vektorové aritmetiky*, z níž se poté generuje logická formule v *CNF* (konjunktivní normální formě). Ta je potom předána *SAT solveru*, který ověří, jestli je daná formule splnitelná. V případě splnitelnosti existuje exekuční cesta v programu s konkrétním ohodnocením proměnných, která porušuje některou specifikovanou vlastnost. Toto ohodnocení je pak protipříkladem k původní hypotéze o správnosti programu.

Nicméně nesplnitelnost formule neimplikuje, že je program – formálněji řečeno, je korektní do hloubky k , což je parametr určující hloubku rozbalení cyklu. To také znamená, že pokud všechny cykly v programu běží nejvíce k iterací, pak nesplnitelnost formule dokazuje plnou korektnost programu vzhledem ke specifikacím. Bounded model checking dokáže analyzovat pouze omezený počet iterací cyklů, a proto nemusí odhalit chyby, které se projeví až po větším počtu opakování. Díky vysoké efektivitě SAT solverů je ale možné cyklus rozbalit na stovky iterací, a proto se dá výsledek této formální verifikace považovat za *dostatečně dobrý*. Je ale zřejmé, že tato technika není dobře škálovatelná pro rozsáhlé projekty o několika desítkách tisíc řádků kódu a více. Výsledná formule je totiž natolik velká, že SAT solver není schopen v *rozumném čase* tuto formuli vyřešit.

V této technické zprávě se zaměřím na vnitřní architekturu CBMC, zejména na způsob překladu programu do *SSA* (static single assignment) reprezentace, bit-vektorové modelování a generování výsledné formule v *CNF*.

2 Technický popis nástroje

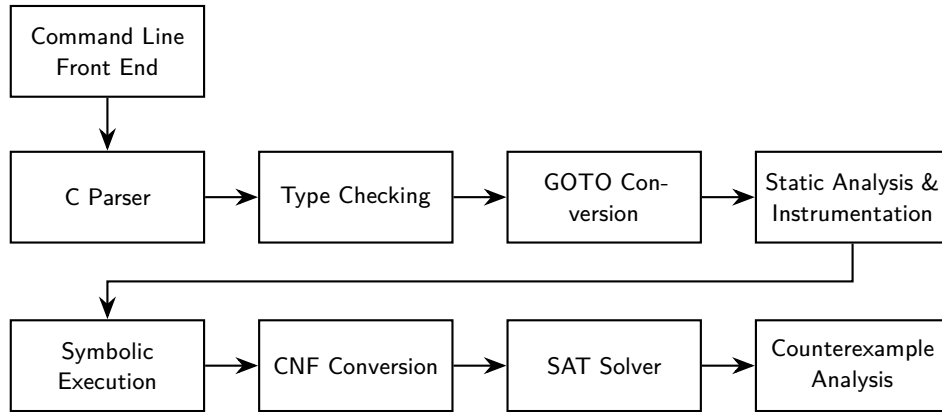
2.1 Předzpracování programu

Než je program převeden do *SSA* reprezentace, musí být provedeno předzpracování. Architektura CBMC má několik vrstev a je ukázána na obrázku 1. CBMC používá *GOTO* programy jako přechodnou (*intermediate*) reprezentaci. V tomto jazyce jsou všechny příkazy *switch*, *if*, cykly a skoky převedeny na ekvivalentní *guarded goto* příkazy. Tento proces probíhá ve fázích *GOTO conversion* a *Static Analysis & Instrumentation* z obrázku 1. Pro jednodušší formální popis zde popíšu převod zmíněných příkazů na *guarded goto* příkazy bez explicitního generování *GOTO* programu, nicméně CBMC tento mezikrok provádí.

2.2 Rozbalení smyček

Rozbalení probíhá následovně:

1. Tělo cyklu je k -krát za sebou vnořené zduplikováno.
2. Řídící příkaz cyklu (*while*, *for*) je v každé kopii nahrazen podmíněným příkazem *if*.
3. Po rozbalení do hloubky k je přidán *unwinding assertion*.



Obrázek 1: Architektura nástroje CBMC (převzato z [1])

První dvě techniky tohoto postupu jsou demonstrovány v programu 1 s fixní hloubkou rozbalení $k = 2$. Formálněji se loop unwinding dá zapsat následující rekurentní definicí. Necht' $W = \text{while}(C)\{B\}$.

$$\text{unwind}(W, k) := \begin{cases} \text{if}(C)\{B; \text{unwind}(W, k-1)\} & \text{pro } k > 0 \\ \text{assert}(\neg C) & \text{pro } k = 0 \end{cases}$$

Na řádce 13 v rozbaleném kódu je již zmíněný unwinding assertion. Tento assert slouží k zaručení korektnosti verifikace. Při převodu na SAT formuli tato podmínka říká, že cyklus byl rozbalen pro všechny iterace, které byly zapotřebí k výpočtu. Splnitelnost formule může odpovídat buď skutečnému porušení specifikace, nebo porušení unwinding assertion, které indikuje nedostatečnou hloubku rozbalení. Intuitivně, pokud se verifikace „nedostane“ k tomuto assertu, pak byly analyzovány všechny potřebné iterace cyklu a analýza je korektní.

2.3 Transformace programu do SSA a konstrukce rovnic

Po předzpracování a rozbalení cyklů se program skládá pouze z podmíněných příkazů `if`, skoků vpřed a assertů. Tento kód je pak převeden na dvě bit-vektorové rovnice:

1. $\mathcal{C}$ (constraints) – množina omezení definujících chování programu.
2. $\mathcal{P}$ (properties) – množina tvrzení, která musí platit.

Proces převodu využívá formu *SSA* (Static Single Assignment), kde je každá proměnná přiřazena nejvýše jednou. Pokud se v programu vyskytuje více zápisů do stejné proměnné v , je nutné rozlišit její jednotlivé verze v čase $(v_1, v_2, \dots)$. Pro tyto účely je zavedena funkce přejmenování $\rho(e)$, kterou využívají všechny kroky převodu programu. Tato funkce rekurzivně prochází výraz e a nahrazuje všechny výskyty proměnných jejich aktuálními verzemi platnými v daném místě programu (např. $\rho(x + y) = x_1 + y_0$).

Program 1: Transformace cyklu `while` s neznámou horní mezí n při fixní hloubce rozbalení $k = 2$.

Původní program	Rozbalený kód ($k = 2$)
<pre>1 int i = 0; 2 int max = 0; 3 4 while (i < n) { 5 if (data[i] > max) { 6 max = data[i]; 7 } 8 9 i++; 10 }</pre>	<pre>1 int i = 0; 2 int max = 0; 3 4 if (i < n) { 5 if (data[0] > max) 6 max = data[0]; 7 i++; 8 9 if (i < n) { 10 if (data[1] > max) 11 max = data[1]; 12 i++; 13 assert(!(i < n)); 14 } 15 }</pre>

2.3.1 Modelování řídicího toku – guards

Dalším krokem je převod dopředných skoků na ekvivalentní podmíněné příkazy `if`, ze kterých jsou posléze vygenerována přiřazení s tzv. *guardy*. Ty reprezentují různé hodnoty, které by se mohly do proměnné přiřadit při předešlém větvení výpočtu.

Eliminace dopředných skoků. Cílem této fáze je odstranit příkaz `goto` tak, že kód, který se má přeskočit, je podmíněn negací podmínky skoku. Ilustrativně, například

```
if (podminka) {
    goto label;
}
prikaz1;
prikaz2;

label:
    prikaz3;
```

je transformováno na ekvivalentní

```
if (!(podminka)) {
    prikaz1;
    prikaz2;
}
prikaz3;
```

Algoritmus pracuje následovně. Uvažujme program P jako sekvenci příkazů. Necht' I je příkaz skoku `goto  $l$` , kde l je návěští definované v programu za příkazem I . Program za příkazem skoku (označme jej např. jako P_{rest}) je rozdělen na tři části vzhledem k návěští l :

$$P_{rest} = \underbrace{\text{blok } X}_{\text{kód k přeskočení}} \quad l : \quad \underbrace{\text{blok } Y}_{\text{zbytek programu}}$$

Samotný příkaz `goto  $l$` je odstraněn. Necht' g je aktuální podmínka, pod kterou se má blok X vykonat. Aby byla zachována sémantika původního skoku, blok X se provede právě tehdy, když se skok neprovedl. Příkazy bloku X jsou tedy obaleny podmínkou $\neg g$:

`if ( $\neg g$ ) { blok  $X$  } blok  $Y$`

Protože program před touto transformací už je zpracován a obsahuje pouze `if` příkazy, g může být komplexní formule obsahující libovolný počet omezení (v praxi to ale bude nanejvýš do několika desítek). Formálně, necht' $e_1, \dots, e_n$ jsou výrazy, které jsou podmínkami, aby se běh programu dostal k aktuálně zpracovávanému skoku `goto  $l$` . Potom:

$$g := \begin{cases} \bigwedge_{i=1}^n e_i & \text{pro } n \geq 1 \\ true & \text{jinak} \end{cases}$$

Tímto procesem se tok řízený skoky převede na tok strukturovaný podmínkami `if`, což je nezbytné pro další krok, kde se podmínky stanou součástí definic proměnných. [2]

2.3.2 Eliminace ukazatelů

Ještě před generováním SSA rovnic a přejmenováním proměnných provádí CBMC eliminaci všech operací dereference ukazatelů. To je nutné kvůli tomu, že výsledná rovnice v bitvektorové logice nepracuje s konceptem paměťových adres, ale jednotlivými proměnnými.

Tento proces je realizován *dereferenční funkcí* $\phi(e, g, o)$, jejímž vstupem je výraz e (ukazatel), aktuálně platný guard g a offset o . Tato funkce transformuje výraz s dereferencí na výraz bez ní, a to tak, že si ke každému ukazateli drží tzv. *points-to* množinu, ve které jsou všichni potenciální kandidáti, na které může daný ukazatel ukazovat. Pro každého z těchto kandidátů se vygeneruje identická sada omezení. Když SAT solver postupně řeší výslednou formuli, dosazování do těchto omezení si můžeme představit jako „výběr“ správného kandidáta. Tento přístup je nadaproximace řešení, což znamená, že nástroj je konzervativní a může vygenerovat formuli, která je splnitelná, ale není reálné, aby takový případ nastal (*false positive*).

V jazyce C jsou dostupné dva typy dereference, a to operátor dereference $*$ a operátor indexování polí $[]$. Funkce ϕ pracuje pro oba operátory stejným způsobem, jen jsou jinak manipulovány offsety. Výraz $*e$ si můžeme představit jako $e[0]$; offset pro operátor $*$ je tedy vždy 0, zatímco pro indexaci pole je to obecně o .

$$*e \longrightarrow \phi(e, g, 0) \quad e[o] \longrightarrow \phi(e, g, o)$$

Funkce ϕ je definována podle toho, jakého typu je výraz e .

1. **Ukazatel:** Necht' p je takový ukazatel. Dosud vygenerovaná rovnice nebo guard g musí obsahovat rovnost $\rho(p) = e'$, kde e' je výraz. Ukazatel p je derefencován aplikací ϕ na e' .

$$\phi(p, g, o) := \phi(e', g, o)$$

2. **Pole:** Necht' a je takové pole. Symbol e je pouze aliasem pro $\&a[0]$.

3. **Adresa jiného symbolu:** Tedy ekvivalent k $e = \&s$. V tomto případě je hodnota $\phi(e, g, o)$ rovna pouze symbolu s , protože dereference adresy symbolu je právě ten symbol. Hodnota offsetu o v tomto případě musí být 0. Navíc je provedena typová kontrola.

$$\phi(\&s, g, o) := s$$

4. **Adresa prvku pole:** Tedy $e = \&a[i]$ pro nějaké pole a a pozici i . Dereference adresy prvku pole $\&a[i]$ je právě ten prvek $a[i]$. Stejně jako v předchozím případě je provedena typová kontrola pro prvky pole.

$$\phi(\&a[i], g, o) := a[i + o]$$

5. **Podmíněný příkaz (ternární operátor):** V takovém případě je funkce ϕ aplikována rekurzivně pro oba případy a podmínka c je přidána do guardu.

$$\phi(c ? e' : e'', g, o) := c ? \phi(e', g \wedge c, o) : \phi(e'', g \wedge \neg c, o)$$

6. **Výraz s ukazatelovou aritmetikou:** Výraz, kde se sčítá ukazatel s nějakým celým číslem. Předpokládejme, že e' je ukazatel a i je celé číslo. Protože i je pouze offset v paměti, je tato hodnota přičtena k offsetu o a funkce ϕ je rekurzivně volána pro výraz e' . Opět je provedena typová kontrola formou `assertu`.

$$\phi(e' + i, g, o) := \phi(e', g, o + i)$$

7. **Přetypování ukazatelů:** Výraz typu $(T*)e'$, kde T je nějaký typ. Funkce ϕ je aplikována na výraz e' , typové kontroly jsou pokryty v ostatních případech, tudíž není třeba žádnou další přidávat.

$$\phi((T*)e', g, o) := \phi(e', g, o)$$

8. **Jiný typ výrazu:** Dle autorů v [3] pro jiné výrazy (v ANSI-C, pro které je tento nástroj implementován) není definována sémantika, a proto nástroj do množiny omezení (bude popsáno v následující sekci) přidává `assert`, že se sem kód nesmí dostat (tedy `assert(¬g)`).

Situaci lze demonstrovat na jednoduchém příkladu. Mějme ukazatel p , který byl někde před aktuálním bodem v programu definován a nástroj zjistil, že může ukazovat na proměnné $\{v_1, v_2, \dots, v_n\}$. Tato množina je právě zmiňovanou *points-to* množinou. Pro zjednodušení řekněme, že proměnné $v_1, \dots, v_n$ jsou primitivního datového typu. Funkce ϕ s *points-to* množinami pracuje následovně.

Nástroj narazí na výraz $*p$. Pokud je tento výraz na pravé straně, máme jednodušší případ. Stačí si pouze ověřit, na kterou proměnnou vlastně aktuálně ukazatel ukazuje a dle toho vybrat správnou hodnotu. Tím pádem bude hodnota funkce ϕ podmíněným výrazem, kde se budou brát v potaz všechny možnosti proměnných z *points-to* množiny.

$$\phi(*p, g, 0) := (p == \&v_1) ? v_1 : ((p == \&v_2) ? v_2 : (\dots))$$

Pokud je ukazatel použit na levé straně přiřazení (např. $*p = x$), nástroj to vyřeší podobným způsobem, protože přesně neví, kterou hodnotu aktualizovat. Nejjednodušší způsob je toto přiřazení vyřešit rozkladem na podmíněné příkazy pro všechny kandidáty z *points-to* množiny. Původní přiřazení $*p = x$ je tedy nahrazeno sekvencí:

```
if (p == &v1) { v1 = x; }
else if (p == &v2) { v2 = x; }
...
```

Každá z těchto větví je následně zpracována standardním způsobem, jako bylo uvedeno na začátku této sekce, tedy rekurzivní aplikací funkce ϕ na jednotlivé příkazy.

2.3.3 Generování omezení a vlastností

Po normalizaci řídicího toku a analýze ukazatelů je možné generovat rovnice zmíněné na začátku této kapitoly, $\mathcal{C}$ a $\mathcal{P}$. Tento proces se provádí pomocí stejnojmenných funkcí $\mathcal{C}(p, g)$ a $\mathcal{P}(p, g)$. Obě dvě jsou definovány dle aktuálního p .

Pokud p je podmíněný příkaz s podmínkou c , **then**-blokem I a **else**-blokem I' , pak obě dvě funkce jsou rekurzivně aplikovány pro oba dva bloky. Pro I , $\rho(c)$ je přidáno do guardu, pro I' je přidáno $\neg\rho(c)$. Výsledky zanoření rekurze jsou pak spojeny konjunkcí.

$$\begin{aligned}\mathcal{C}(\text{if } (c) \ I \ \text{else } I', g) &:= \mathcal{C}(I, g \wedge \rho(c)) \wedge \mathcal{C}(I', g \wedge \neg\rho(c)) \\ \mathcal{P}(\text{if } (c) \ I \ \text{else } I', g) &:= \mathcal{P}(I, g \wedge \rho(c)) \wedge \mathcal{P}(I', g \wedge \neg\rho(c))\end{aligned}$$

Pro sekvenci příkazů jsou funkce také definovány rekurzivně. Necht' p je sekvence příkazů složená z I a I' . Potom jsou obě funkce rekurzivně aplikovány na I a I' následujícím způsobem:

$$\begin{aligned}\mathcal{C}(I; I', g) &:= \mathcal{C}(I, g) \wedge \mathcal{C}(I', g) \\ \mathcal{P}(I; I', g) &:= \mathcal{C}(I, g) \wedge \mathcal{P}(I', g)\end{aligned}$$

Necht' p je tvrzení (*assertion*) s argumentem a . Tento argument je přejmenován funkcí ρ a vrácen jako vlastnost programu, která se bude ověřovat. Střežen je guardem g . Funkce $\mathcal{C}$ s tvrzeními nedělá nic, respektive přidá *true* do konjunkce.

$$\mathcal{P}(\text{assert}(a), g) := g \implies \rho(a)$$

Přiřazení jsou také přidávána jako omezení do C po přejmenování proměnných. Pokud proměnná v má být přepsána hodnotou nového výrazu e pod guardem g , pak pokud g

platí, proměnná je přepsána novým výrazem a pokud ne, proměnná si zachovává stejnou hodnotu. Tento problém je řešen „ternárním operátorem“, který bere ohled na daný guard.

$$\mathcal{C}(v = e, g) := v_\alpha = g ? \rho(e) : v_{\alpha-1}$$

Intuitivně, guard g nemůže být vyhodnocen během transformace, tudíž musí být součástí tohoto omezení.

Pro pole hodnot je konstrukce těchto omezení komplikovanější. Mějme příkaz $v[a] = e$. Hodnota v poli v_α na indexu i je rovno $\rho(e)$ pokud guard g platí a $i = a$, jinak $v_\alpha[i] = v_{\alpha-1}[i]$. Pole jsou modelována λ -funkcemi:

$$\mathcal{C}(v = e, g) := v_\alpha = \lambda i : (g \wedge i = \rho(a)) ? \rho(e) : v_{\alpha-1}[i]$$

Intuitivně vysvětleno, pole je funkce, která pro daný index i vrátí $\rho(e)$, tedy novou hodnotu přepsanou výrazem e , pokud $i = \rho(a)$ (a samozřejmě musí platit guard g), jinak vrací hodnotu ze starého pole $v_{\alpha-1}[i]$. Do formule s tvrzeními musí být přidána kontrola mezí pole.

$$\mathcal{P}(v = e, g) := 0 \leq \rho(a) \wedge \rho(a) < \text{len}(v_\alpha)$$

Dalším problémem jsou strukturované datové typy. Autoři v [4] zmiňují, že struktury jsou v podstatě *syntactic sugar* pro pole. Nicméně během analýzy známe velikost struktury a offsety jednotlivých položek struktury. Proto není třeba reprezentovat struktury pomocí λ -funkce, ale můžeme do nich přistupovat statickým indexem, respektive odkazy na jednotlivé členy. Necht' s je proměnná typu **struct** a příkaz přiřazení je $s.f = e$, kde f je člen této struktury. Potom nová verze struktury s_α je pro každý člen m definována následovně:

$$\mathcal{C}(s.f = e, g) := \begin{cases} g ? \rho(e) : s_{\alpha-1}.f & \text{pro } m = f \\ s_{\alpha-1}.m & \text{pro } m \neq f \end{cases}$$

2.4 Bit-vektorová aritmetika

V předchozí kapitole byly definovány rovnice omezení a vlastností ($\mathcal{C}$ a $\mathcal{P}$) nad proměnnými programu. Tyto proměnné jsou běžně chápány jako proměnné s neomezenou doménou; to ale neplatí v hardwaru. Každá proměnná v jazyce C má pevnou bitovou šířku, protože jazyk C je nízkoúrovňový jazyk, který se překládá do strojového kódu a tím pádem musí respektovat architekturu, na které je zdrojový kód přeložen. Všechny proměnné jsou tedy reprezentovány jako *bitové vektory* pevné šířky w . Díky této reprezentaci může nástroj hledat chyby související s bitovou aritmetikou, jako jsou například *přetečení*, *podtečení*, chyby spojené s reprezentací čísel... Všechny operace se dají zapsat jako operace nad množinami zbytků po dělení 2^w , jedná se tedy o modulární aritmetiku (při použití vhodné reprezentace, např. dvojkový doplněk). Například sčítání:

$$a +_w b = (a + b) \pmod{2^w}$$

CBMC tohoto faktu využívá a pro zdrojový program staví jeho reprezentaci ve formě ekvivalentního elektrického obvodu, který provádí primitivní operace (sčítání, apod.). CBMC řeší otázku „Existuje nastavení vstupních bitů tak, aby výstup byl '1'?“ (tedy existuje ohodnocení proměnných tak, že $\mathcal{C} \wedge \neg \mathcal{P}$?). Proto daný obvod musí převést na logickou formuli, kterou potom předá SAT solveru.

2.4.1 Konstrukce obvodů (Flattening)

Každá bit-vektorová proměnná x o šířce w je rozložena na vektor w výrokových proměnných $x_0, \dots, x_{w-1}$. Aritmetické a relační operátory jsou následně nahrazeny ekvivalentními hardwarovými obvody.

Jako příklad uvažujme sčítání $s = a + b$. CBMC pro tuto operaci vygeneruje logický obvod odpovídající tzv. *Ripple Carry Adder*. Tento obvod se skládá z kaskády úplných sčítaček. Pro každý bit i jsou vygenerovány výrokové formule definující výsledný bit s_i a přenos do vyššího řádu c_{i+1} :

$$\begin{aligned} s_i &\iff a_i \oplus b_i \oplus c_i \\ c_{i+1} &\iff (a_i \wedge b_i) \vee (c_i \wedge (a_i \oplus b_i)) \end{aligned}$$

Kde $\oplus$ značí operaci XOR. Obdobně jsou konstruovány obvody pro odčítání, porovnávání (sčítačky, negace), bitové posuny (multiplexory), násobení, apod. Tato technická zpráva ilustruje techniku flatteningu na jednoduchém příkladu sčítání; její podrobný popis je uveden v [4, kap. 6.2].

2.4.2 Převod do CNF

Výsledek konstrukce obvodu je orientovaný acyklický graf reprezentující ten daný zkonstruovaný obvod. Vrcholy tohoto grafu jsou logická hradla, hrany mezi nimi jsou dráty mezi hradly. Tento graf reprezentuje formuli:

$$\Phi = \mathcal{C} \wedge \neg \mathcal{P}$$

Pro použití moderních SAT solverů je nutné převést tento graf do *konjunktivní normální formy* (CNF). Přímý převod by mohl vést k exponenciálnímu nárůstu velikosti formule, proto se využívá *Tseitinova transformace* [5]. To je transformace původní formule φ na *ekvisplnitelnou* formuli ψ , což znamená, že φ je splnitelná právě tehdy, když ψ je splnitelná. Jinými slovy, existuje model formule φ právě tehdy, když existuje model formule ψ . Tyto modely ale mohou být libovolné a nemusí pro obě formule existovat stejný počet modelů, aby byla zachována ekvisplnitelnost.

Tseitinova transformace. Pro lepší intuici uvažujme jednoduchý logický obvod reprezentující výraz $z = (a \wedge b) \vee c$. Jak bylo zmíněno, algebraické převody přes distributivní zákony nejsou proveditelné. Tseitinova transformace místo toho zavádí pomocnou proměnnou pro každý mezivýsledek (výstup hradla).

Nechť w_1 je pomocná proměnná reprezentující výstup hradla AND ($a \wedge b$). Původní výraz se tím rozpadne na dvě jednoduché rovnice (lokální omezení):

$$\begin{aligned} w_1 &\iff (a \wedge b) \\ z &\iff (w_1 \vee c) \end{aligned}$$

Každá z těchto ekvivalencí je poté standardním algebraickým převodem převedena na sadu klauzulí:

$$\begin{aligned} w_1 &\iff (a \wedge b) \equiv (w_1 \implies (a \wedge b)) \wedge ((a \wedge b) \implies w_1) \\ &\equiv (\neg w_1 \vee (a \wedge b)) \wedge (\neg(a \wedge b) \vee w_1) \\ &\equiv (\neg w_1 \vee a) \wedge (\neg w_1 \vee b) \wedge (\neg a \vee \neg b \vee w_1) \end{aligned}$$

Stejným způsobem se zpracuje výraz $z \iff (w_1 \vee c)$, kde w_1 reprezentuje výsledek konjunkce v původním výrazu $z = (a \wedge b) \vee c$.

$$z \iff (w_1 \vee c) \equiv (\neg w_1 \vee z) \wedge (\neg c \vee z) \wedge (w_1 \vee c \vee \neg z)$$

Výsledná SAT formule je konjunkcí všech šesti klauzulí. Protože klauzule pro konkrétní typy hradel jsou vždy stejné, nemusíme složitě algebraicky počítat převody ekvivalencí do CNF. Místo toho můžeme mít například tabulky, kde pro konkrétní hradlo najdeme výraz v CNF, který můžeme do výsledné formule přidat. Výhoda Tseitinovy transformace je maximálně lineární nárůst délky formule.

Formule je následně předána SAT solveru. Pokud solver nalezne splnitelné ohodnocení (SAT), znamená to, že existuje kombinace vstupních bitů, která vede k porušení vlastnosti $\mathcal{P}$ a z modelu solveru je rekonstruován protipříklad. V opačném případě (UNSAT) je program (do hloubky k) korektní.

3 Demonstrace základních vlastností CBMC

V této sekci se budeme věnovat experimentům, které demonstrují schopnosti i limity nástroje CBMC. Nejprve představíme sadu jednoduchých, cíleně zvolených příkladů, jejichž cílem je ilustrovat, jak CBMC detekuje různé typy chyb (přetečení, chyby práce s pamětí, apod.) a jakým způsobem prezentuje nalezené protipříklady.

Druhá část sekce je věnována řadícího algoritmu Merge sort. Tento algoritmus je výrazně komplexnější než úvodní příklady, protože využívá rekurzi a dynamickou práci s poli. Na této úloze demonstrujeme použitelnost nástroje CBMC na složitějším algoritmu a diskutujeme jeho omezení, která vyplývají z techniky bounded model checkingu.

3.1 Demonstrace funkcionality na syntetických příkladech

V [repozitáři](#) autorů nástroje CBMC je rozsáhlá sada jednoduchých testů, které slouží především k regresivnímu testování nástroje. Tyto příklady jsou ale záměrně velmi minimalistické a jejich podstatou není ilustrovat chování nástroje na realističtějších programech. Proto jsem se nakonec rozhodl napsat si několik jednoduchých kódů v jazyce C, z nichž každý má přibližně 20 řádků. Každý z těchto příkladů je navržen tak, aby demonstroval určitou vlastnost nebo slabinu CBMC na stále jednoduchém, ale srozumitelném kódu, pro který se dá vysvětlit jeho praktické využití.

3.1.1 Příklad 1 – Sekvence podmínek vedoucí k porušení vlastnosti

Cílem tohoto příkladu je demonstrovat jednu z hlavních sil model checkingu kombinovaného se SAT solvingem, konkrétně schopnost automaticky nalézt takovou kombinaci vstupních hodnot, která vede k porušení sledované vlastnosti programu, a to i v přítomnosti složitě zanořených podmínek. Kód v programu 2 obsahuje jednoduchou funkci, kde po sekvenci podmínek je z funkce vrácena hodnota -1 , která reprezentuje chybu. Funkce `nondet_int()` reprezentuje nedeterministické hodnoty, například vstupy od uživatele do programu.

Program 2: Porušení `assertu` po splnění několika komplexních podmínek.

```
1  #include <assert.h>
2
3  int nondet_int();
4  int complex_condition(int a, int b)
5  {
6      if (a > 1000 && b > 1000) {
7          if (a == b + 53) {
8              if ((a % 2 == 0) && (b % 2 != 0)) {
9                  return -1;
10             }
11         }
12     }
13     return 0;
14 }
15
16 int main()
17 {
18     int x = nondet_int();
19     int y = nondet_int();
20     int result = complex_condition(x, y);
21     assert(result != -1);
22     return 0;
23 }
```

Ačkoliv je uvedený příklad uměle zjednodušený, obdobné situace se mohou vyskytovat i v reálných programech. Takové podmínky pravděpodobně nebudou hned za sebou v jedné funkci, mohou být stovky řádků kódu od sebe. Splnění určité kombinace podmínek může vést například k neoprávněnému zvýšení práv (může vést na útok *privilege escalation*). Díky SAT solveru se najde přesné ohodnocení proměnných `a` a `b`, které poruší `assert` na řádce 20.

CBMC po spuštění s tímto zdrojovým programem vygeneruje očekávaný výstup:

```
[main.assertion.1] line  assertion result != -1: FAILURE
```

Tato chyba ovšem není jediná v uvedeném zdrojovém kódu. Může totiž dojít k přetečení při sčítání operandů `b` a `53` ve funkci `complex_condition(int a, int b)`, výsledek tedy může být chybný. Této problematice se budeme věnovat v následujícím příkladu.

3.1.2 Příklad 2 – přetečení výsledku operace

Celočíselné přetečení demonstrujeme na kódu, který je typický například pro implementaci binárního vyhledávání v řazeném poli. Jedná se o výpočet středu celočíselného intervalu¹.

Program 3: Chybný výpočet hodnoty poloviny intervalu $\langle l, h \rangle$.

```

1  int nondet_int();
2  int main()
3  {
4      int l = nondet_int();
5      int h = nondet_int();
6      int m = (l + h) / 2;
7      return 0;
8  }
```

CBMC při analýze programu detekuje přetečení při sčítání proměnných `l` a `h`:

```
[main.overflow.1] line 6 arithmetic overflow on signed + in l + h: FAILURE
```

Tato chyba je velmi zrádná, protože běžné testování ji často vůbec neodhalí. Při použití menších datových typů pro indexy polí se může projevit již během testování, avšak pro 32bitová celá čísla se s vysokou pravděpodobností objeví až po nasazení. Chybu lze odstranit záměnou $\frac{l+h}{2}$ za $l + \frac{h-l}{2}$ (je ale třeba zaručit $h \geq l$). V druhém případě se při výpočtu středu nedostaneme mimo interval $\langle l, h \rangle$ při výpočtu.

3.1.3 Příklad 3 – přístup mimo meze pole

Tento příklad demonstruje přístup mimo meze pole způsobený nedostatečnou kontrolou iterační proměnné. Podobná situace může v praxi vzniknout například při práci s daty, jejichž velikost je získána z externího zdroje (uživatelský vstup, soubor, síťová komunikace,...) a její hodnota není dostatečně ověřena.

V uvedeném kódu v programu 4 není hodnota proměnné `n` nijak omezena shora. Pokud `n` bude mít hodnotu větší než velikost pole `arr`, dojde k zápisu mimo jeho meze, což je v jazyce C nedefinované (pro dynamicky alokované pole často vede k chybě přístupu do paměti, např. *segmentation fault*).

V reálných programech se chyba může vyskytnout například při mazání částí polí nebo zpracování pouze části vstupních dat, kdy se implicitně předpokládá, že vstupní hodnota nepřekročí velikost alokovaného úložiště. Tato chyba je lépe testovatelná než přetečení

¹Tento pojem není technicky zcela přesný; pro účely tohoto textu však budeme celočíselný interval chápat jako množinu $\{l, l+1, \dots, h\}$.

Program 4: Zápis mimo meze pole způsobený neověřenou hodnotou proměnné *n*.

```
1  int nondet_int();
2
3  int main()
4  {
5      int arr[5];
6
7      for (int i = 0; i < 5; i++) {
8          arr[i] = nondet_int();
9      }
10
11     int n = nondet_int();
12     if (n > 0) {
13         for (int i = 0; i < n; i++) {
14             arr[i] = 0;
15         }
16     }
17
18     return 0;
19 }
```

celočíselných operandů, nicméně testování mezních hodnot může být lehce přehlédnuto. CBMC je schopen tuto chybu detekovat díky použití SAT solveru, který ověřuje korektnost přístupu do pole pro všechny možné hodnoty proměnné *n*.

Bez explicitního omezení rozbalení smyček není CBMC schopen analýzu dokončit. Důvodem je skutečnost, že jsme nespecifikovali, jak hluboko se mají rozbalit smyčky. Pro náš případ stačí přidat přepínač `--unwind` s hodnotou 6, protože víme, že chyba nastane po 6 iteracích. Obecně by tato hodnota měla odpovídat velikosti pole, do kterého se přistupuje. Po spuštění správného příkazu dostáváme výsledek:

```
[main.overflow.2] line 13 arithmetic overflow on signed + in i + 1: UNKNOWN
[main.unwind.1] line 13 unwinding assertion loop 1: FAILURE
[main.array_bounds.3] line 14 array 'arr' lower bound in arr[(signed long int)i]:
UNKNOWN
[main.array_bounds.4] line 14 array 'arr' upper bound in arr[(signed long int)i]:
FAILURE
```

Vidíme, že nástroj našel chybu při poslední iteraci přístupu do pole. Navíc není nijak ošetřeno celočíselné sčítání v inkrementaci indexu *i*, což vede k potenciálnímu přetečení výrazu *i + 1*. CBMC dokáže odvodit, že hodnota proměnné *i* roste od 0 do *n*, ale bez znalosti horní meze *n* není schopen jednoznačně rozhodnout o bezpečnosti inkrementace, proto generuje upozornění (UNKNOWN).

3.1.4 Příklad 4 – Analýza ukazatelů

V této sekci budeme demonstrovat schopnosti CBMC v rámci ukazatelové analýzy. Zatím se omezíme na paměť alokovanou na zásobníku a budeme se věnovat chybné dereferenci ukazatele. Práci s dynamicky alokovanou pamětí si představíme v následující sekci. Kód pro tento experiment je sepsán v programu 5.

Program 5: Jednoduchý příklad demonstrující nevalidní dereferenci ukazatele.

```
1  #define FLAG_INIT 0x1
2  #define FLAG_PROCESS 0x2
3  int nondet_int();
4  int main() {
5      int flags = nondet_int();
6      int *ptr = NULL;
7      int buffer = 0;
8
9      if (flags & FLAG_INIT) {
10         ptr = &buffer;
11     }
12
13     if (flags & FLAG_PROCESS) {
14         *ptr = 100;
15     }
16
17     return 0;
18 }
```

Tento kód demonstruje příklad z programování nízkoúrovňových systémů. Stejně jako v předchozích příkladech, dvě podmínky pracující s ukazatelem `ptr` mohou být v kódu od sebe vzdáleny. Vývojář může předpokládat, že pokud je nastaven druhý příznakový bit, může data bezpečně zpracovat, protože byla inicializována. Ta ale v tomto případě proběhnout nemusela a programátor velmi jednoduše může dereferencovat nulový ukazatel.

CBMC pro tento příklad generuje očekávané zprávy:

```
[main.pointer_dereference.1] line 19 dereference failure: pointer NULL in *ptr:
FAILURE
[main.pointer_dereference.2] line 19 dereference failure: dead object in *ptr:
UNKNOWN
[main.pointer_dereference.3] line 19 dereference failure: pointer outside object
bounds in *ptr: FAILURE
```

První chyba je jasná, jedná se o dereferenci nulového ukazatele.

Upozornění `dead object in *ptr` odkazuje na možnost dereference ukazatele, který neukazuje na platný a živý objekt. V tomto konkrétním příkladu se nejedná o klasický

dangling pointer vzniklý uvolněním paměti, ale o situaci, kdy CBMC není schopen jednoznačně prokázat, že ukazatel `ptr` vždy odkazuje na validní objekt. Chyba je UNKNOWN kvůli tomu, že CBMC tuto vlastnost nedokáže v dané konfiguraci jednoznačně ověřit.

Třetí chyba je vygenerována kvůli interní reprezentaci NULL ukazatele v CBMC. V jazyce C je NULL reprezentací nulového ukazatele, zatímco CBMC jej interně reprezentuje jako nevalidní referenci. Nástroj tento přístup vyhodnocuje jako přístup na offset 0 k objektu, který neexistuje, a proto generuje chybu, že bylo přistoupeno mimo paměť alokovanou pro objekt.

3.1.5 Příklad 5 – práce s dynamickou pamětí

Poslední ze syntetických příkladů se zabývá prací s dynamickou pamětí. Uvažujme implementaci nějakého síťového protokolu. V tomto protokolu pracujeme s datovou strukturou `Session`. Na počátku inicializujeme tuto datovou strukturu, z neznámého zdroje dostaneme číslo události, kterou máme provést. Provedení dané události je reprezentováno funkcí `do_something()`. Kompletní kód je vypsán v programu 6.

Spuštění CBMC na tomto kódu generuje velký výstup, protože pro dynamickou paměť se provádí velké množství kontrol, a to například kontroly *use-after-free*, *double free*, argumentů funkce `free()`, memory leaks, apod. Navíc, tento kód je rozsáhlejší než ostatní příklady a CBMC provádí kontroly pro každou tuto funkci. Nás samozřejmě bude nejvíce zajímat funkce `do_something()`. Tato funkce je hlavní implementací protokolu, pro ilustraci postačí uvažovat jeden zápis do objektu `*s`.

Po spuštění CBMC dostáváme výstup:

```
[do_something.pointer_dereference.1] line 11 dereference failure: pointer NULL in
s->status: SUCCESS
[do_something.pointer_dereference.2] line 11 dereference failure: pointer invalid
in s->status: SUCCESS
[do_something.pointer_dereference.3] line 11 dereference failure: deallocated
dynamic object in s->status: FAILURE
[do_something.pointer_dereference.4] line 11 dereference failure: dead object in
s->status: UNKNOWN
[do_something.pointer_dereference.5] line 11 dereference failure: pointer outside
object bounds in s->status: UNKNOWN
[do_something.pointer_dereference.6] line 11 dereference failure: invalid integer
address in s->status: UNKNOWN
```

První dvě kontroly ověřují možnost dereference nulového nebo obecně nevalidního ukazatele `s`. Třetí kontrola pro zapsání do dealokovaného objektu na řádce 11. Při bližším prozkoumání kódu zjistíme, že na řádce 18 ve funkci `handle_event()` byl uvolněn objekt `Session`, nicméně nijak to nebylo volajícímu sděleno a ten dále předpokládal, že je objekt stále validní. Jak je v kódu naznačeno, programátor nejspíše zapomněl vrátit hodnotu indikující, že objekt byl uvolněn a neměl by se dále používat. Poslední tři kontroly jsou důsledkem práce s již dealokovaným objektem, protože z tohoto bodu není CBMC schopen jednoznačně rozhodnout o dalších vlastnostech ukazatele. Ukazatel `s` je ale stejně neplatný a díky tomu by mohl být problém dopočítat další kontroly.

Program 6: Ukázka chyby *use-after-free* při práci s dynamickou pamětí.

```
1  typedef struct
2  {
3      int id;
4      int status;
5  } Session;
6
7  int nondet_int();
8
9  void do_something(Session* s)
10 {
11     s->status = 1;
12 }
13
14 int handle_event(Session* s, int event)
15 {
16     if (event == 404) {
17         free(s);
18         // forgot to return err value
19     }
20     return 0;
21 }
22
23 int main()
24 {
25     Session* s = (Session*)malloc(sizeof(Session));
26     if (!s)
27         return 0;
28
29     s->id = 1;
30     s->status = 0;
31
32     int event = nondet_int();
33     if (handle_event(s, event) != 0) {
34         exit(EXIT_FAILURE);
35     }
36
37     do_something(s);
38     return 0;
39 }
```

3.2 Případová studie – Merge sort

V předchozí sekci jsme se soustředili na to, jak CBMC řeší izolované jednoduché problémy. Tato sekce bude o analýze řadícího algoritmu Merge sort, který je pro bounded model checking problematický ze dvou důvodů:

1. Je rekurzivní, tudíž musí proběhnout unwind zanořených volání do sebe.
2. Pracuje s poli, což vede k výraznému nárůstu počtu proměnných ve výsledné SAT formuli.

3.2.1 Implementace algoritmu a verifikační cíle

Kód algoritmu je uveden v příloze A v programu 9. Pro účely analýzy byla zvolena klasická rekurzivní implementace algoritmu, který rozděljuje pole na dvě poloviny, tyto poloviny rekurzivně seřadí a následně je sloučí do původního pole. Pro verifikační účely máme definovanou proměnnou `SIZE`, která určuje velikost pole, které se bude řadit. Budeme zkoumat časy běhu analýzy pro různé velikosti tohoto pole. Pole je na začátku běhu programu naplněno nedeterministickými hodnotami, aby nástroj byl donucen ověřit všechny možné hodnoty v poli.

Cílem verifikace je dokázat korektnost implementace. Zaměříme se především na práci s poli, řešení rekurze a správnost řazení algoritmu. V rámci následujících sekcí se také pokusíme do algoritmu záměrně zanést chyby, které algoritmus znehodnotí a nebude poté řadit správně. Takto znehodnocený algoritmus budeme také analyzovat.

3.2.2 Nastavení CBMC a parametry analýzy

CBMC vyžaduje parametr `--unwind` pro každý rekurzivní vstup, jinak bude rozbalovat smyčky, případně rekurze, donekonečna. Výhodou algoritmu Merge sort je jeho deterministická povaha dělení dat. Při rekurzivním zanoření vždy rozdělí pole na dvě poloviny, což znamená, že počet rekurzivních zanoření bude shora omezen výrazem $\lceil \log_2(\text{SIZE}) \rceil$. Parametr `--unwind` se ale v tomto případě vztahuje jak na rekurzivní volání funkce `Merge_sort`, tak na smyčky použité při slučování dílčích polí. Při slučování posledního pole proběhne v nejhorším případě řádově `SIZE` iterací, neboť dvě podpole jsou sloučena do jednoho výsledného pole původní velikosti. Tato hodnota však sama o sobě nestačí, protože kromě provedení všech iterací musí být cyklus rozbalen ještě jednou navíc, aby mohl být korektně ověřen tzv. unwinding assertion, tedy že smyčka skutečně doběhla pro všechny přípustné iterace. Protože $\text{SIZE} + 1 \geq \lceil \log_2(\text{SIZE}) \rceil$ a zároveň $\text{SIZE} + 1$ bezpečně pokrývá maximální počet iterací jednotlivých smyček, je hodnota $\text{SIZE} + 1$ použita jako parametr hloubky rozbalení pro CBMC.

Dále CBMC nijak modifikovat nebudeme; zanecháme všechny standardní kontroly. Naším cílem není optimalizovat běh programu pro určité typy úloh, ale sledovat, jak se nástroj chová ve výchozím nastavení, jak by mohl být použit pro reálné úlohy.

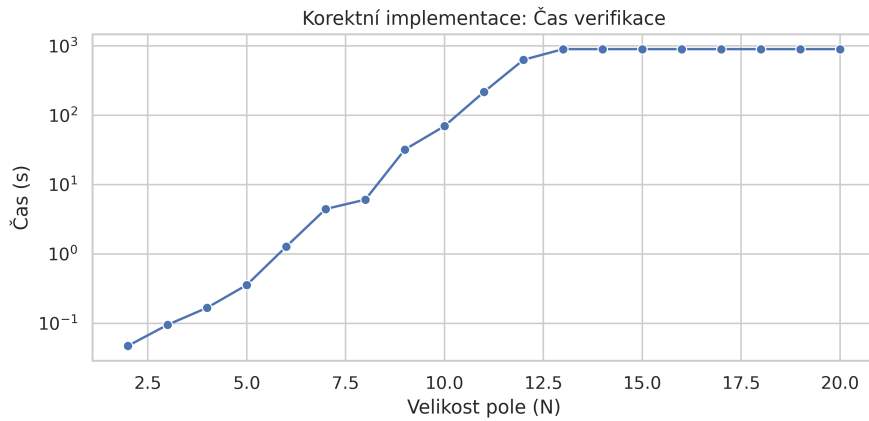
Nezapomeňme (jak bylo zmiňováno v sekci 2), že provedená analýza bude dokazovat korektnost pouze pro konkrétní hodnoty `SIZE`. Nepředstavuje důkaz korektnosti algoritmu pro libovolné vstupy.

Velikost vstupního pole má zásadní vliv na běh programu, protože výsledná SAT formule velmi rychle roste. Proto ponecháme všechny standardní kontroly zapnuté a budeme se soustředit na rozdíly v experimentech při zvyšování velikosti pole.

3.2.3 Experimenty s původním algoritmem

Pro účely této technické zprávy bylo provedeno celkem 18 běhů původního nezměněného programu s různými velikostmi pole. Struktura algoritmu Merge sort je pro nástroj CBMC nevhodná, což vede k vysoké časové náročnosti analýzy. Budeme zkoumat především rychlost verifikace. Dále si ukážeme, jak velké byly vygenerované CNF formule a budeme diskutovat časový poměr mezi generováním formule a jejím řešením.

Rychlost verifikace. Na obrázku 2 je znázorněna závislost času verifikace na velikosti vstupního pole. Vertikální osa je v logaritmickém měřítku, přičemž je patrné, že s rostoucí



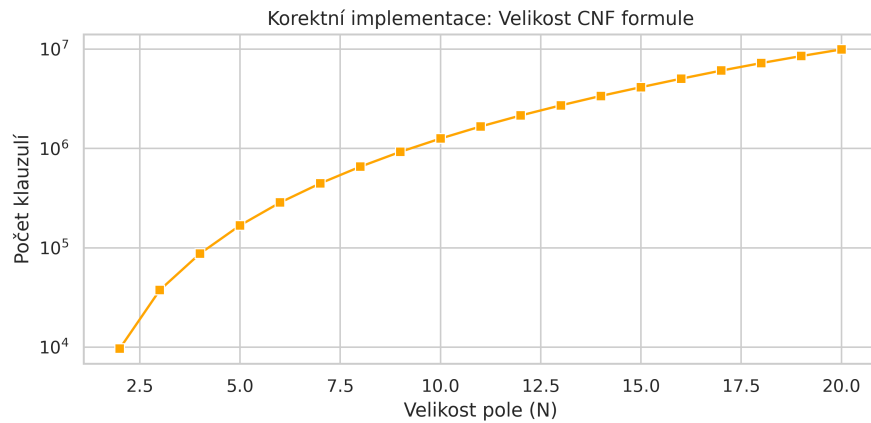
Obrázek 2: Graf závislosti rychlosti verifikace na velikosti pole při verifikaci standardní implementace algoritmu Merge sort.

velikostí pole čas verifikace velmi rychle narůstá. Při spouštění analýzy byl nastaven timeout 15 minut; pro hodnoty $SIZE \geq 13$ již CBMC nebyl schopen korektnost programu v daném časovém limitu ověřit.

Tento výsledek ilustruje praktické omezení použití bounded model checkingu pro rekurzivní algoritmy pracující s poli (respektive pracující s jakýmkoliv modelem, díky kterému verifikovaný model explozivně narůstá), a to i pro malé velikosti vstupů.

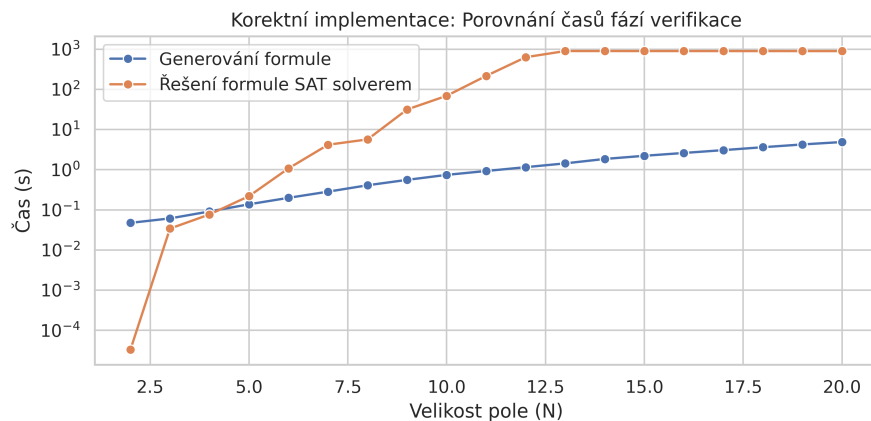
Velikost CNF formule. Na obrázku 3 je znázorněna velikost CNF formule generované CBMC v závislosti na velikosti vstupního pole. I zde je vertikální osa v logaritmickém měřítku. Velikost formule roste velmi rychle a pro $SIZE = 20$ se již pohybuje v řádu milionů klauzulí. Nárůst velikosti formule je zřetelný zejména pro menší hodnoty $SIZE$, kde každé zvětšení vstupu znamená výrazné rozšíření výsledné formule.

Samotná velikost formule však ještě plně nevysvětluje extrémní nárůst času verifikace, což je patrné z následujícího rozboru jednotlivých částí verifikace.



Obrázek 3: Graf závislosti velikosti vygenerované CNF formule na velikosti pole při verifikaci standardní implementace algoritmu Merge sort.

Časová složitost různých fází verifikace. Obrázek 4 porovnává čas strávený generováním formule pro SAT solver a čas potřebný k jejímu řešení. Pro malé velikosti pole



Obrázek 4: Graf srovnání časové složitosti fáze generování formule pro SAT solver a jejím řešením při verifikaci standardní implementace algoritmu Merge sort.

($SIZE \leq 4$) je dominantní fází generování formule, zatímco samotné řešení SAT problému je relativně levné. S rostoucí velikostí vstupu se však situace rychle obrací a řešení formule se stává jednoznačně nejnáročnější částí verifikace.

Tento jev odpovídá očekávání: zatímco generování formule roste přibližně lineárně s velikostí modelu, složitost řešení SAT problému je výrazně ovlivněna kombinatorickou explozí možných stavů programu.

3.2.4 Experimenty s modifikovaným algoritmem

Do původního Merge sort algoritmu byly postupně vneseny dvě chyby. První chyba spočívala v prohození řadící podmínky na řádce 23, aby algoritmus řadil sestupně a ne vzestupně.

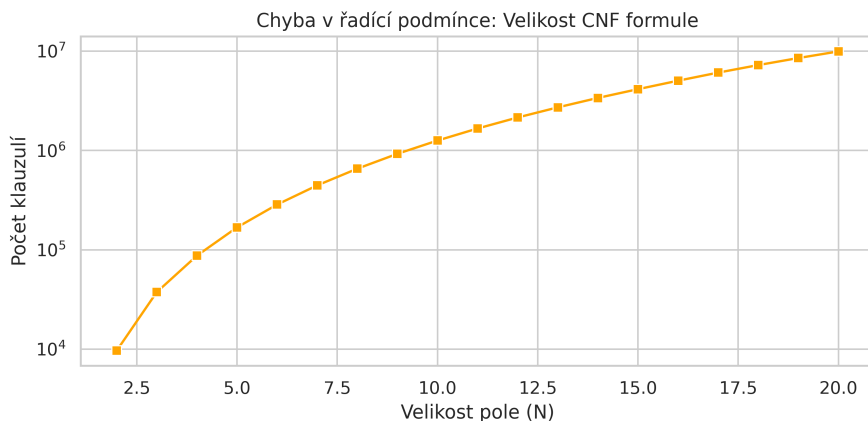
Druhá modifikace simuluje implementační chybu způsobenou opomenutím závěrečné fáze algoritmu, kdy dochází ke zkopírování zbývajících prvků poté, co jedno podpole bylo plně zpracováno. V kódu se jedná o řádky 35–45.

Chyba směru řazení algoritmu. Pro účely prvního experimentu byla vnesena do kódu opačná podmínka pro řazení. Jediná změna tedy proběhla na řádce 23:

```
if (left_subarray[i] <= right_subarray[j])  
    ↓  
if (left_subarray[i] > right_subarray[j])
```

Z hlediska výpočetní náročnosti by se tento experiment neměl lišit od verifikace korektní implementace. Předpokládáme, že SAT formule bude přibližně stejně velká, protože během zpracování kódu se vygenerují přibližně stejné klauzule, jen podmínky v guardech budou opačné pro ekvivalentní if příkazy. Může ale nastat situace, kdy SAT solver najde ohodnocení proměnných vedoucí ke splnitelnosti rychleji než důkaz, že žádná kombinace vstupních proměnných ke splnitelnosti nevede. To ale závisí na vnitřním algoritmu SAT solveru, protože existují různé heuristiky, které tyto nástroje implementují.

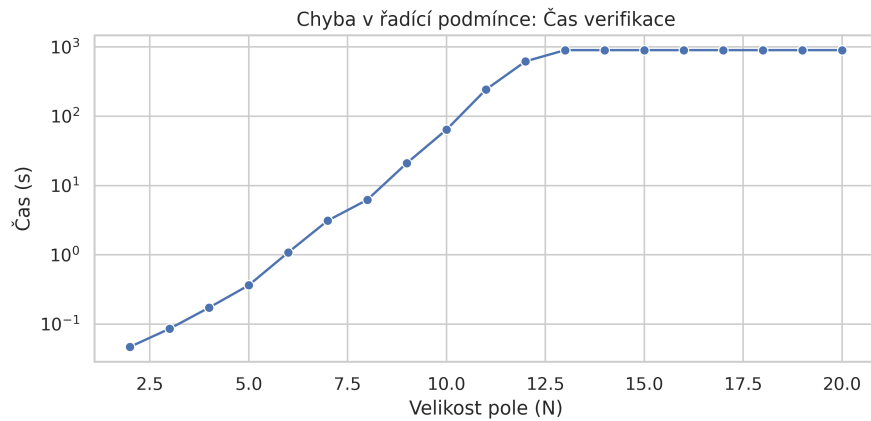
Nejdříve ověříme velikost SAT formule vzhledem k velikosti pole, pro které se kód ověřuje. To je ukázáno na obrázku 5. Vidíme, že velikost generovaných formulí se prakticky



Obrázek 5: Graf závislosti velikosti formule na velikosti pole při obrácení řadící podmínky.

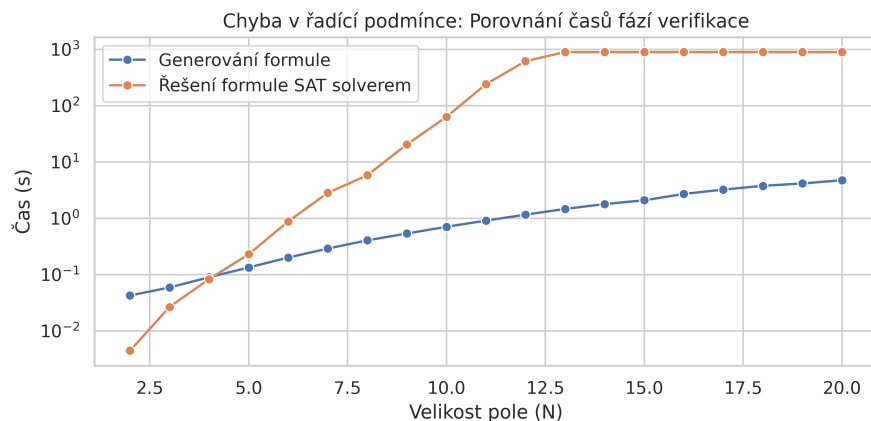
nezměnila, což potvrzuje naši hypotézu. Druhé případné zrychlení by mohlo nastat v situaci, kdy formule má příznivou strukturu pro SAT solver. Časové výsledky verifikace jsou na obrázku 6.

Ani rychlost verifikace se nezměnila, což značí velmi podobnou strukturu formule. Tato chyba tedy verifikaci prakticky nijak neovlivnila. Stále trvá velmi dlouho, pro větší pole se



Obrázek 6: Graf závislosti rychlosti verifikace na velikosti pole při obrácení řadicí podmínky.

tento problém stává prakticky neřešitelným v rozumném čase. Dá se tedy předpokládat, že budou i podobné časové poměry mezi generováním formule a jejím řešením; tato hypotéza je potvrzena na obrázku 7.

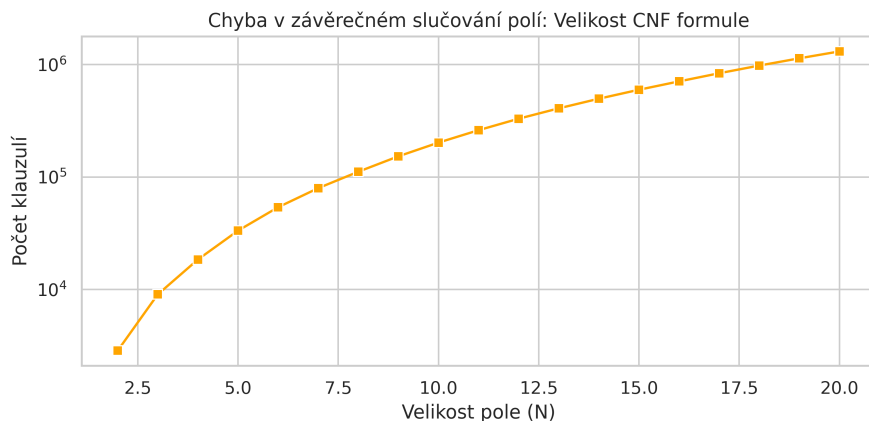


Obrázek 7: Graf srovnání rychlosti fáze generování formule pro SAT solver a jejího řešení při verifikaci Merge sortu s obrácenou řadicí podmínkou.

Chyba opomenutí závěrečné fáze algoritmu. Druhou uměle vnesenou chybou je smazání cyklů na řadcích 35–45. Budeme se především zabývat otázkou, jak se tato změna projeví na době analýzy algoritmu. V případě korektní implementace nebo prohození směru řazení analýza trvala přibližně stejnou dobu, a to neúnosně dlouhou. Druhá modifikace je větší zásah do struktury algoritmu. Odstranění závěrečných slučovacích smyček vede k tomu, že podpole vzniklá slučováním jsou zjevně nekorektní, což by se mohlo projevit i na průběhu verifikace. CBMC neposuzuje sémantickou zjevnost chyby, ale generuje SAT formuli, která program reprezentuje. Ta bude menší, protože jsou odstraněny dva cykly, které by se jinak rozbalovaly při každém rekursivním volání funkce `Merge()`. Výsledná

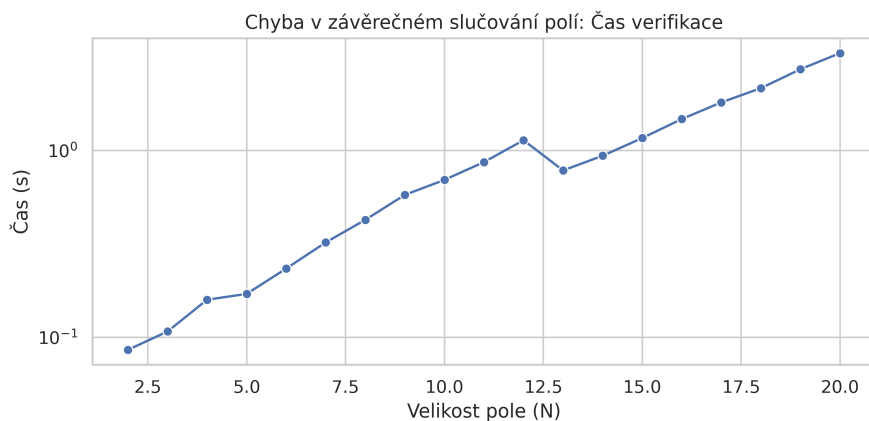
SAT formule tím pádem obsahuje méně proměnných i klauzulí a její řešení může být pro SAT solver snazší. Nicméně doba analýzy stále záleží na řešitelnosti dané formule.

Pro druhou modifikaci algoritmu proto očekáváme zrychlení verifikace. Nejprve si ověříme, že generovaná formule je opravdu menší. Velikosti formulí vůči velikostem ověřovaných polí jsou na obrázku 8.



Obrázek 8: Graf závislosti velikosti formule na velikosti pole při odstranění závěrečných slučovacích smyček ve funkci `Merge()`.

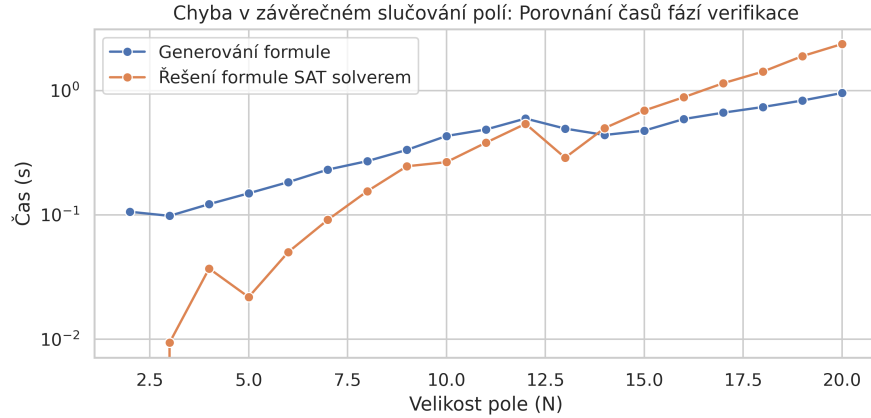
Generované formule jsou opravdu řádově menší. Vzhledem k exponenciálnímu nárůstu výpočetní složitosti řešení SAT formule v závislosti na její velikosti můžeme předpokládat, že doba verifikace se také řádově sníží. Výsledek je na obrázku 9.



Obrázek 9: Graf závislosti rychlosti verifikace na velikosti pole při odstranění závěrečných slučovacích smyček ve funkci `Merge()`.

Vidíme, že rychlost verifikace se zvýšila dokonce o několik řádů a všechny verifikované instance byly vyřešeny relativně rychle. Dokonce si můžeme povšimnout, že pole o velikosti 13 bylo vyřešeno rychleji než pole o velikosti 12, což značí, že struktura formule byla pro SAT solver příznivější a byl schopen ji rychleji vyřešit.

Tento experiment má zajímavé výsledky i v rámci porovnání rychlosti generování formule a jejím řešením, protože jednotlivé instance formulí jsou SAT solverem vyřešeny poměrně rychle. Graficky jsou časy znázorněny na obrázku 10.



Obrázek 10: Graf srovnání rychlosti fáze generování formule a jejího řešení při verifikaci algoritmu Merge sort s opomenutým slučováním zbytků polí.

3.2.5 Diskuze omezení a škálovatelnosti

V sekci 3.2 jsme ověřovali, jak funguje technika bounded model checkingu pro algoritmus reálného charakteru. Pro experimenty byl záměrně zvolen algoritmus Merge sort, protože je rekurzivní a pracuje s pomocnými poli, přičemž obě tyto programovací techniky jsou pro bounded model checking velmi komplikované na ověřování vlivem exploze stavového prostoru. Verifikovali jsme algoritmus v korektní implementaci a také algoritmy modifikované s vnesenými dvěma chybami. Výsledky verifikací byly očekávané, tedy pro korektní algoritmus verifikace hlásila úspěch, pro nekorektní algoritmy hlásila neúspěch.

Verifikace tohoto algoritmu trvala velmi dlouhou dobu, jak jsme si ukázali v grafech. Kvůli explozi stavového prostoru se velikost generovaných formulí exponenciálně zvyšovala a pro relativně malá pole formule obsahovaly i několik milionů klauzulí.

Z toho plyne, že tato technika je obtížně aplikovatelná pro rozsáhlé softwarové systémy, které mohou mít miliony řádků kódu. Tento nástroj byl ale vyvinut pro verifikaci vestavěných systémů a kritických komponent, kde je kladen extrémní důraz na korektnost implementace. Bounded model checking s jistotou ověří, že daný program neobsahuje chyby v rámci uvažovaného stavového prostoru (hloubka rozbalení k). Proto je používán pro verifikaci stěžejních modulů a segmentů kódu, jejichž spolehlivost je klíčová. Tohoto paradigmatu se budeme držet i v následující kapitole.

4 Experimenty s CBMC na reálných kódech

V této kapitole se zaměříme na verifikaci vybraných školních projektů. Jak bylo zmíněno v předešlé kapitole, aplikace CBMC na rozsáhlejší softwarová díla vyžaduje pečlivý výběr

verifikovaných komponent kvůli výpočetní náročnosti. Navíc nástroj nativně nepodporuje systémová volání jako jsou například funkce `scanf()`, jejichž chování je potřeba modelovat, aby nástroj věděl, jakých hodnot mohou nabývat proměnné, které s těmito funkcemi pracují.

V této kapitole budeme verifikovat dva projekty:

1. **Druhý projekt předmětu IZP**, který spočíval v implementaci relační a množinové kalkulačky – práce s textovým vstupem a jeho parsování.
2. **Projekt předmětu IMP**, ve kterém byla implementována jednoduchá kalkulačka ovládaná pomocí rotačního enkodéru na mikrokontroléru – embedded logika a stavový automat.

4.1 Verifikace implementace relační a množinové kalkulačky

Tento program je konzolová aplikace implementovaná v jazyce C. Slouží k provádění základních operací nad množinami a binárními relacemi. Pracuje ve dvou fázích – nejdříve načte vstupní soubor, poté pracuje s inicializovanými datovými strukturami, nad kterými provádí operace.

Data získaná z řádků jsou uložena v poli. Každý prvek pole (řádek) je struktura, která nese informaci o typu a obsahu daného řádku:

1. Univerzum – výčet všech řetězců ve formě jednosměrného seznamu, se kterými program dále pracuje.
2. Množina – výčet řetězců na daném řádku ve formě jednosměrného seznamu, každý z nich musí být v univerzu.
3. Relace – jednosměrný seznam dvojic prvků ve formátu (x, y) , každý prvek dvojice musí být v univerzu.
4. Příkaz – instrukce pro provedení operace nad relacemi nebo množinami.

Vstupní soubor může vypadat například následovně:

U a b c	(Definice univerza {a, b, c})
S a b	(Definice množiny A = {a, b})
S b c	(Definice množiny B = {b, c})
C union 2 3	(Příkaz: Sjednocení množin na radku 2 a 3)
C equals 2 3	(Příkaz: Test rovnosti množin na radku 2 a 3)

Následuje fáze interpretace příkazů, po dokončení vypíše výsledek na standardní výstup, případně chyby na standardní chybový výstup. V kontextu verifikace se budeme zabývat logickou správností implementace funkcí, které provádějí operace nad množinami a samotné parsování souboru ve verifikační části vynecháme, respektive pro CBMC naimplementujeme funkce, které vrací nedeterministické řetězce, množiny a relace. Dále se budeme zabývat prací s dynamickou pamětí při manipulaci se seznamy.

4.1.1 Konstrukce verifikačního prostředí

CBMC zná funkce `nondet_int()`, `nondet_char()`, a podobné, které slouží pro generování nedeterministických hodnot primitivních datových typů. Pro naše účely tyto hodnoty nestačí, protože program může pracovat obecně s libovolnými řetězci v množinách. Tato skutečnost velmi výrazně limituje použití CBMC pro verifikaci tohoto programu, protože povolení dlouhých řetězců by znamenalo kombinatorickou explozi stavového prostoru a verifikace by nebyla uskutečnitelná.

Jako doménu pro obsah řetězců jsme tedy zvolili znaky 'a' až 'e'. Délka řetězců je v kódu parametrizována makrem `STR_LEN`. Pro tuto konkrétní analýzu je nastavena na 1, protože cílem je verifikovat především logiku množinových operací a potenciální kolize. Implementace generátoru využívá primitivum `__CPROVER_assume`, které omezuje stavový prostor solveru pouze na validní znaky z naší domény. Konkrétní implementace je v programu 7.

Program 7: Implementace instrumentovaného generování nedeterministických řetězců při analýze relační a množinové kalkulačky.

```
1 void nondet_string(char* string)
2 {
3     for (unsigned i = 0; i < STR_LEN; i++) {
4         char c = nondet_char();
5         __CPROVER_assume(c >= 'a' && c <= 'e');
6
7         string[i] = c;
8     }
9     string[STR_LEN] = '\0';
10 }
```

Dále byly instrumentovány funkce pro generování náhodných množin a relací – ve smyčce volají funkci pro generování řetězců, které jsou postupně do dané množiny/relace přidávány.

4.1.2 Specifikace verifikovaných vlastností

Pro experiment byly vybrány celkem 4 operace nad množinami a relacemi. Konkrétně se jedná o množinové sjednocení a průnik, u relací byl zvolen symetrický uzávěr a výpis oboru hodnot (domény) relace.

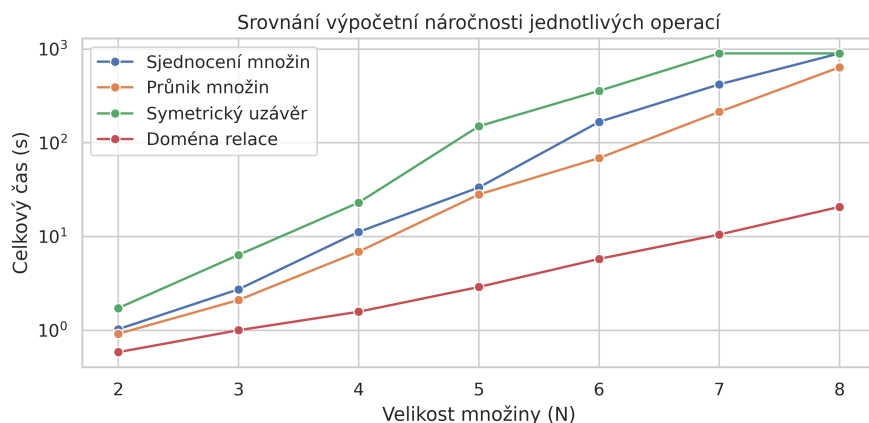
Kromě verifikace funkcionality (která je řešena pomocí `assertů` a pomocných verifikačních funkcí kontrolujících platnost hledané vlastnosti) budeme verifikovat i standardní bezpečnostní vlastnosti, které CBMC kontroluje automaticky (práce s pamětí, dereference ukazatelů apod.).

Pro experiment byly zvoleny velikosti struktur (množin a relací) od 2 do 8 prvků. Měření byly stejné metriky jako u algoritmu Merge sort v kapitole 3.2.

4.1.3 Výsledky analýzy

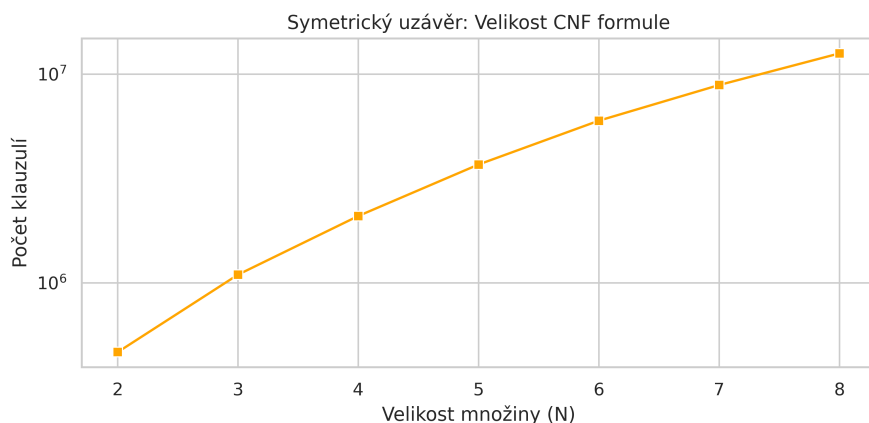
Během všech 24 běhů analýzy nástroj CBMC nenašel žádnou chybu. To indikuje, že pro dané operace, zvolené velikosti množin a omezené univerzum je implementace s vysokou pravděpodobností korektní.

Z hlediska výpočetní náročnosti se jednotlivé operace výrazně lišily. Souhrnné porovnání časové náročnosti všech verifikovaných operací je zobrazeno na obrázku 11.



Obrázek 11: Srovnání časové náročnosti verifikace jednotlivých operací v závislosti na velikosti vstupních dat.

Časově nejnáročnější operací byl výpočet symetrického uzávěru. Jak je patrné z grafu 11, pro velikosti množin 7 a 8 verifikace dosáhla časového limitu 15 minut a nebyla dokončena. Důvodem je extrémní nárůst velikosti generované SAT formule. Vztah mezi velikostí vstupu a počtem klauzulí ve výsledné formuli pro tuto operaci ukazuje obrázek 12.



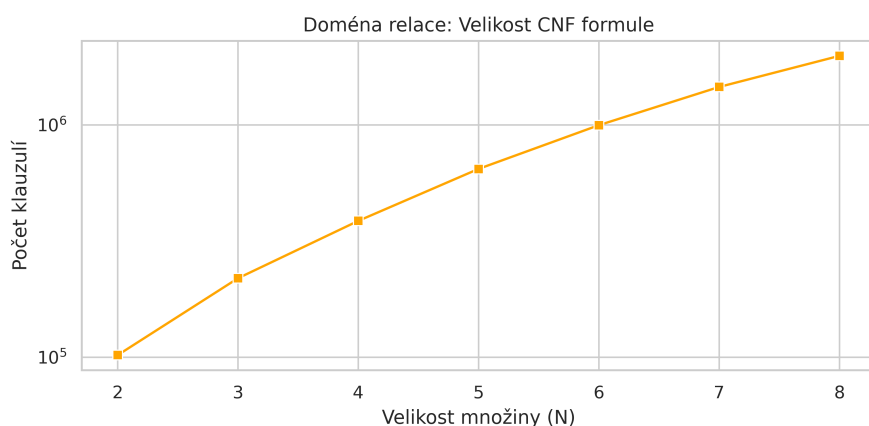
Obrázek 12: Exponenciální nárůst počtu klauzulí SAT formule pro operaci symetrického uzávěru.

Zatímco pro malé vstupy se pohybujeme v řádu statisíců klauzulí, pro množinu o velikosti 8 dosáhla výsledná formule velikosti 12 milionů klauzulí. Takový objem dat již SAT solver

nedokázal v rozumném čase zpracovat.

Operace sjednocení a průniku množin vykazovaly podobnou náročnost. U průniku byly všechny instance úspěšně verifikovány, i když poslední instance se limitu blížila. Sjednocení množin bylo výpočetně mírně náročnější a verifikace největší instance o velikosti 8 skončila vypršením časového limitu. Velikosti formulí se zde pohybovaly od nižších stovek tisíc do jednotek milionů klauzulí.

Nejméně náročnou operací bylo hledání domény relace. Verifikace proběhla velmi rychle, v řádu desítek sekund i pro největší instance. Tomu odpovídá i velikost formulí, kde poslední dvě instance jen lehce přesáhly jeden milion klauzulí, což je vidět na obrázku 13. I zde je



Obrázek 13: Exponenciální nárůst počtu klauzulí SAT formule pro operaci vyhledání domény relace.

však zřejmý exponenciální trend nárůstu stavového prostoru, ačkoliv absolutní čísla jsou řádově nižší než u symetrického uzávěru.

4.2 Verifikace implementace kalkulačky ovládané rotačním enkodérem

Druhým analyzovaným projektem je firmware pro mikrokontrolér (konkrétně Kinetis K60), který provádí funkci jednoduché kalkulačky. Je ovládána rotačním enkodérem, který má zároveň funkcionalitu tlačítka. Logika programu je rozdělena na obsluhu hardwarových vstupů a samotnou aplikační logiku.

Pro CBMC je největší problém závislost programu na konkrétním hardwaru, protože program přímo přistupuje ke konkrétním registrům. Aby analýza byla možná, je možné tyto registry simulovat nedeterministickými hodnotami nebo izolovat čistě algoritmické části kódu. V rámci tohoto experimentu se zaměříme na druhou metodu a budeme verifikovat ukládání nových výsledků operací do cyklického bufferu, konkrétně budeme zkoumat přístup k tomuto poli.

4.2.1 Analýza cyklického bufferu a detekce chyby

Modul `result_array.c` implementuje cyklický buffer fixní velikosti 5, do kterého se ukládají nově spočítané výsledky. Implementace je pomocí indexu `i`, který se s novým prvkem

dekrementuje a po dosažení indexu 0 je zresetován zpět na nejvyšší index v bufferu. Cílem verifikace tohoto modulu je ověření, že kód správně přistupuje do pole a nedochází k zápisu mimo vyhrazenou paměť. Nicméně po spuštění CBMC byla zjištěna závažná chyba, a to *off-by-one* přístup do nealokované části paměti.

Funkce `InsertResult` implementující přidávání prvků do pole je v programu 8. Chyba

Program 8: Funkce `InsertResult` implementující vkládání nově vypočtených výsledků do cyklického pole.

```
1 void InsertResult(CalcData* result)
2 {
3     static int i = PREVIOUS_RESULTS_SIZE - 1;
4     newestResult = i;
5     lastResults[i] = *result;
6     i--;
7     if (i == 0) {
8         i = PREVIOUS_RESULTS_SIZE;
9     }
10    return;
11 }
```

je při analýze kódu zjevná. Na řádce 8 se po dosažení indexu 0 vrací buffer zpět na velikost pole. Velikost pole je sice 5, ale ihned v dalším volání této funkce je přistoupeno mimo rozsah alokovaného pole na šestou položku. V důsledku nízké komplexnosti programu a malému využití zásobníku se chyba během implementace a používání vůbec neprojevila. Je to ale zákeřná chyba, kvůli které se mohou omylem přepsat 4 byty dat, jež mohou být kriticky důležité, především ve světě vestavěných systémů. CBMC tuto chybu najde a přesně řekne, kde se nachází a jakých hodnot mají proměnné nabývat. Výpis 1 ukazuje část trace vygenerované nástrojem CBMC, která demonstruje překročení hranic pole.

```
State 152 file Sources/result_array.c function InsertResult line 42 thread 0
-----
i=0 (00000000 00000000 00000000 00000000)

State 154 file Sources/result_array.c function InsertResult line 44 thread 0
-----
i=5 (00000000 00000000 00000000 00000101)

Violated property:
file Sources/result_array.c function InsertResult line 41 thread 0
array 'lastResults' upper bound in lastResults[(signed long int)i]
!((signed long int)i >= 51)
```

Výpis 1: Výpis nástroje CBMC prokazující zápis mimo pole.

Z výpisu 1 je patrné, že v okamžiku, kdy proměnná `i` dosáhne hodnoty 0 ve stavu 152,

dojde k vykonání chybné logiky pro resetování indexu. Následně ve stavu 154 má proměnná hodnotu 5, což vede k porušení podmínky horní meze pole. Ve správné implementaci by na řádku 8 mělo být správné přiřazení `i = PREVIOUS_RESULTS_SIZE - 1`.

Tento experiment ukazuje sílu statické analýzy, která dokáže odhalit chyby, které se dynamickým testováním odhalují velmi obtížně. Při testování by programátor pravděpodobně nezaznamenal žádné neobvyklé chování, ale při nasazení do produkce by chyba tohoto typu mohla způsobit pád kritického systému.

4.2.2 Verifikace stavového automatu

V druhém experimentu se zaměříme na verifikaci řídicí logiky stavového automatu. Cílem je nalézt takovou sekvenci vstupních signálů z rotačního enkodéru, která by uvedla systém do nevalidního nebo neošetřeného stavu. Konkrétně budeme ověřovat návratovou hodnotu funkce obsluhující stavový automat.

Tento experiment demonstruje schopnost CBMC abstrahovat od hardwarové vrstvy. Místo čtení hodnot z fyzických registrů, které nejsou při statické analýze dostupné, instrumentujeme kód tak, aby vracel nedeterministické hodnoty simulující chování reálného hardwaru. Ukázka této instrumentace je k dispozici v příloze B v programech 10 a 11.

Korektní stavový automat může vrátit jednu z následujících hodnot:

- `ENCODER_CW` nebo `ENCODER_CCW` při detekci otočení po/proti směru hodinových ručiček,
- `ENCODERX_BTN_PRESSED`, kde $X \in \{1, 2\}$, při stisku tlačítka jednoho z enkodérů,
- `CLEAR_BTN` při detekci resetovacího tlačítka.

Verifikační podmínkou je, že po libovolné sekvenci interakcí musí funkce vrátit validní hodnotu z této množiny, nebo zůstat ve stavu `IDLE`, pokud žádná interakce není zaznamenána.

Analýza byla spuštěna pro sekvence interakcí o délce maximálně 200 kroků. Zajímavým zjištěním tohoto experimentu je výrazný nepoměr mezi dobou předzpracování a samotným řešením formule. Celkový čas běhu byl přibližně 74 s, přičemž 72 s zabralo předzpracování a generování formule a pouhé 2 sekundy trvalo řešení SAT problému. Ačkoliv vygenerovaná formule obsahovala přes 1,5 milionu klauzulí, její logická struktura byla pravděpodobně pro SAT solver triviální, což vedlo k velmi rychlému vyřešení.

Výsledek experimentu potvrdil, že logika stavového automatu je s vysokou pravděpodobností implementována korektně. Vzhledem k tomu, že zpracování jednoho fyzického otočení enkodérem trvá maximálně 4 iterace automatu a následné interakce jsou na těch předchozích nezávislé, zvolená hloubka verifikace (200 iterací) dostatečně pokrývá stavový prostor potřebný k odhalení případných chyb v logice přechodů.

5 Závěr

Cílem této technické zprávy byla podrobná analýza nástroje CBMC, který využívá techniku *bounded model checking* pro formální verifikaci programů v jazyce C.

V kapitole 2 jsme se věnovali vnitřní architektuře nástroje a jeho formálním vlastnostem. Popsali jsme proces předzpracování vstupního programu, jeho převod do interní reprezentace a následné generování ekvisplnitelné SAT formule, která je poté řešena SAT solverem.

Kapitola 3 demonstrovala funkčnost nástroje z praktického hlediska. Na sadě syntetických příkladů jsme ukázali schopnost CBMC detekovat chyby, jejichž odhalení pomocí dynamického testování bývá obtížné. Druhá část kapitoly se věnovala řadícímu algoritmu *Merge sort*, na kterém jsme ilustrovali limity této techniky. Experimenty potvrdily extrémní nárůst výpočetní náročnosti i pro velmi malé vstupy.

V kapitole 4 jsme se zaměřili na aplikaci nástroje na reálné kódy z vybraných univerzitních projektů. Vzhledem k vysoké výpočetní náročnosti jsme verifikovali pouze klíčové komponenty těchto programů. Komplexní formální verifikace celých projektů by byla pro rozumně nastavenou hloubku rozbalení časově neproveditelná.

Tato zpráva tak poskytuje ucelený přehled o možnostech nástroje CBMC, včetně zhodnocení jeho výhod a omezení při verifikaci složitějších softwarových systémů.

Zdroje

1. KROENING, Daniel; TAUTSCHNIG, Michael. CBMC–C Bounded Model Checker: (Competition Contribution). In: *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 2014, s. 389–391.
2. CLARKE, Edmund; KROENING, Daniel. Hardware verification using ANSI-C programs as a reference. In: *Proceedings of the 2003 Asia and South Pacific Design Automation Conference*. 2003, s. 308–311.
3. CLARKE, Edmund; KROENING, Daniel; YORAV, Karen. *Behavioral Consistency of C and Verilog Programs Using Bounded Model Checking*. Pittsburgh, PA, 2003-05. Technical Report, CMU-CS-03-126. School of Computer Science, Carnegie Mellon University.
4. KROENING, Daniel; STRICHMAN, Ofer. *Decision procedures*. Sv. 1. Springer, 2016.
5. TSEITIN, Grigori S. On the complexity of derivation in propositional calculus. In: *Automation of reasoning: 2: Classical papers on computational logic 1967–1970*. Springer, 1983, s. 466–483.

A Kód řadícího algoritmu Merge sort

Program 9: Kód řadícího algoritmu Merge sort analyzovaného v sekci 3.2.

```
1  #define SIZE 5
2
3  int nondet_int();
4
5  void Merge(int arr[], int left, int mid, int right)
6  {
7      int i, j, k;
8      int left_subarray_size = mid - left + 1;
9      int right_subarray_size = right - mid;
10     int left_subarray[left_subarray_size], right_subarray[right_subarray_size];
11
12     // Copy data to temp arrays L[] and R[]
13     for (i = 0; i < left_subarray_size; i++)
14         left_subarray[i] = arr[left + i];
15     for (j = 0; j < right_subarray_size; j++)
16         right_subarray[j] = arr[mid + 1 + j];
17
18     i = 0;
19     j = 0;
20     k = left;
21     // Merge back into arr[l..r]
22     while (i < left_subarray_size && j < right_subarray_size) {
23         if (left_subarray[i] <= right_subarray[j]) {
24             arr[k] = left_subarray[i];
25             i++;
26         }
27         else {
28             arr[k] = right_subarray[j];
29             j++;
30         }
31         k++;
32     }
33
34     // Copy the remaining elements of L[], if any are left
35     while (i < left_subarray_size) {
36         arr[k] = left_subarray[i];
37         i++;
38         k++;
39     }
40     // Copy the remaining elements of R[], if any are left
41     while (j < right_subarray_size) {
42         arr[k] = right_subarray[j];
43         j++;
44         k++;
45     }
46 }
```

Program 9: Kód řadícího algoritmu Merge sort – pokračování

```
47 void merge_sort(int arr[], int left, int right)
48 {
49     if (left < right) {
50         int mid = left + (right - left) / 2;
51
52         merge_sort(arr, left, mid);
53         merge_sort(arr, mid + 1, right);
54
55         Merge(arr, left, mid, right);
56     }
57 }
58
59 int main()
60 {
61     int arr[SIZE];
62
63     for (int i = 0; i < SIZE; i++) {
64         arr[i] = nondet_int();
65     }
66
67     merge_sort(arr, 0, SIZE - 1);
68
69     for (int i = 0; i < SIZE - 1; i++) {
70         assert(arr[i] <= arr[i + 1]);
71     }
72
73     return 0;
74 }
```

B Instrumentace kódu pro verifikaci stavového automatu

Program 10: Instrumentovaná hlavička souboru `fsm.c` implementující logiku stavového automatu. Jsou vytvořeny falešné registry, které jsou poté v rámci hlavní smyčky naplněny nedeterministickými hodnotami na začátku každé iterace.

```
1  #ifndef CBMC_VERIFICATION
2  // mock hardware pro cbmc
3  #include <stdint.h>
4  int nondet_int();
5
6  typedef struct
7  {
8      uint32_t PDIR;
9  } GPIO_Type;
10
11  GPIO_Type mock_pta;
12  GPIO_Type mock_pte;
13
14  GPIO_Type* PTA = &mock_pta;
15  GPIO_Type* PTE = &mock_pte;
16
17  # define FSM_LOOP_LIMIT 200
18  #else
19  // realny hardware
20  # include "MK60D10.h"
21  #endif
```

Program 11: Část původního kódu implementující stavový automat využívající „nové“ registry definované pro analýzu. Navíc je uveden limit pro počet iterací cyklu, který reprezentuje maximální délku posloupnosti jednotlivých akcí, které uživatel provede s rotačním enkodérem.

```
1  #ifndef CBMC_VERIFICATION
2  // Verifikace -- omezeny pocet kroku
3  for (int iter = 0; iter < FSM_LOOP_LIMIT; iter++) {
4      PTA->PDIR = nondet_int();
5      PTE->PDIR = nondet_int();
6  #else
7  // Puvodni nekonecna smycka pro mikrokontroler
8  while (1) {
9      #endif
10      a = !(PTA->PDIR & (1 << pin_a));
11      b = !(PTA->PDIR & (1 << pin_b));
12      btn = !(PTA->PDIR & (1 << pin_btn1));
13      clr_btn = !(PTE->PDIR & (1 << pin_clr_btn));
```
